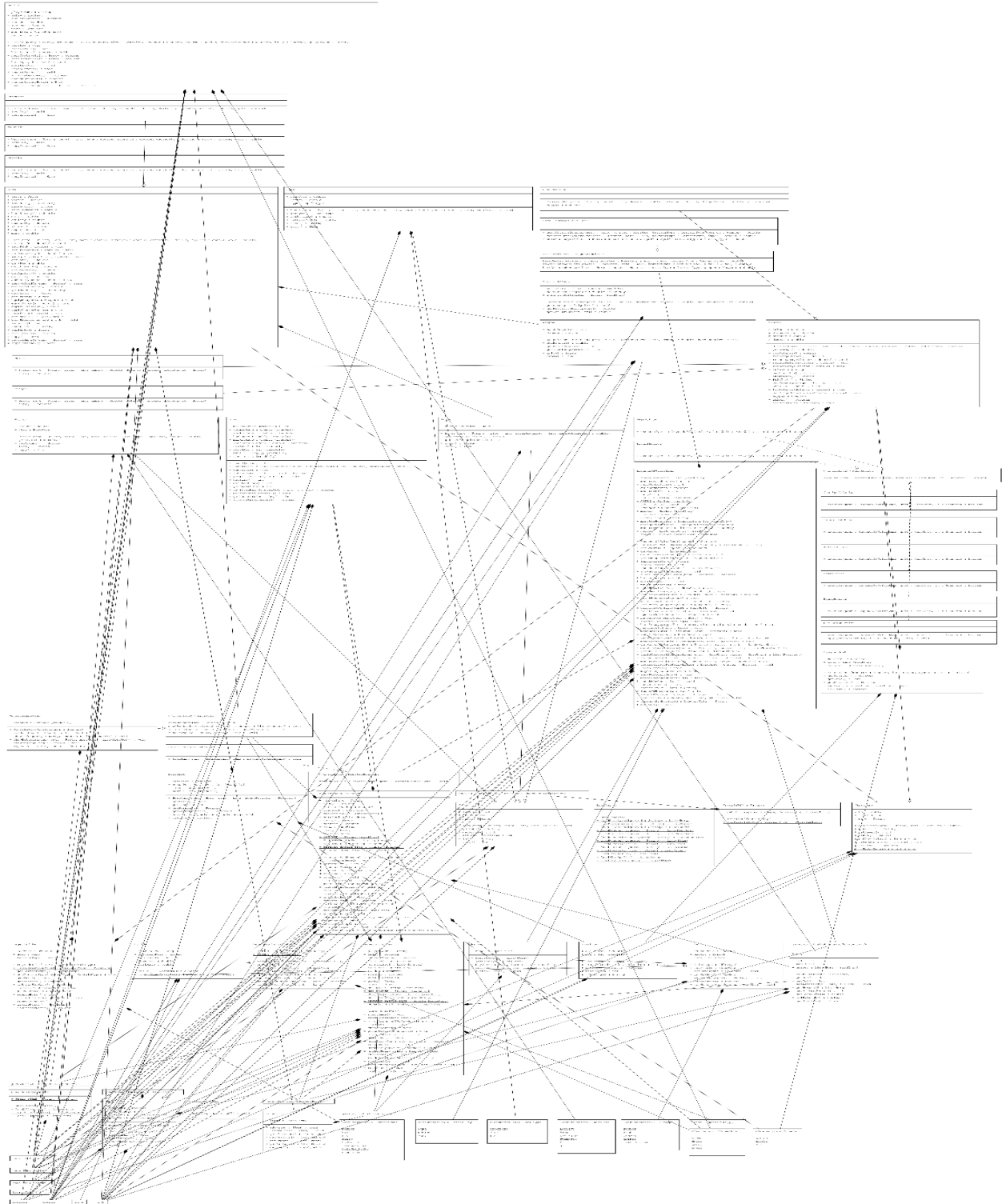


Design Documentation of Legends of Valor

Ankita Patra, U38177365

Xiju Jiang, U03732023

Ying Huang, U13787608



1) Classes:

i) Core Game Abstraction

Game (abstract)

- Defines generic game lifecycle (startGame)
- Enforces game loop structure
- Declares hooks: initializeGame, heroesTurn, monstersTurn, checkWinCondition, endGame

ii) Game Implementations

MonstersAndHeroesGame

- Implements classic grid-based RPG gameplay
- Handles movement, markets, battles, weather, day/night
- Uses Board, Party, Battle
- Extends Game

LegendsOfValorGame

- Implements lane-based MOBA-style gameplay
- Manages heroes, monsters, lanes, and nexus rules
- Handles combat via actions and AI
- Extends Game

(iii) Board & World

Board

- Square grid for Monsters & Heroes
- Tile generation and monster tracking
- Start position and bounds checking

LegendsOfValorBoard

- Fixed 8×8 board with lanes and walls
- Nexus placement
- Terrain-based rules
- Independent from the Board

Tile / LegendsTile

- Represents a single cell
- Stores terrain, monster, hero, market

(iv) Characters

Hero

- Player-controlled character
- Stats, inventory, equipment, and leveling

Monster

- Enemy entity
- Combat stats and behavior

HeroUnit

- Wraps Hero with board position and lane

MonsterUnit

- Wraps Monster with board position and lane

Party

- Group of heroes
- Shared movement and battle participation

(v) Combat & Actions

Battle

- Turn-based combat engine (Monsters & Heroes)

Action classes (MoveAction, AttackAction, etc.)

- Encapsulate a single hero action
- Follow the Command/Strategy pattern
- Used only in Legends of Valor

DamageCalculator

- Computes damage using terrain, equipment

MonsterBehavior

- AI logic for monster turns

(vi) Items & Economy

Item (abstract)

- Base class for all items

Weapon, Armor, Potion, Spell

- Concrete item types with combat effects

Inventory

- Stores hero items

Market

- Buy/sell logic
- Shared by both games

(vii) UI & Utilities

GameView

- Abstract UI interface

ConsoleGameView

- Console-based rendering and messaging

Enums

- TileType, TerrainType, Lane, Weather, etc.
- Represent fixed game rules

Class Relationships

1) Inheritance

- Game → MonstersAndHeroesGame
- Game → LegendsOfValorGame
- Item → Weapon, Armor, Potion, Spell

MonstersAndHeroesGame and LegendsOfValorGame extend the abstract Game base class. Ensures a common game lifecycle (initializeGame, heroesTurn, monstersTurn, checkWinCondition, endGame) across multiple game modes.

Template Method Pattern – Game.startGame() defines the fixed algorithm structure, while subclasses provide specific behavior.

2) Composition / Has-A

- MonstersAndHeroesGame → Board, Party, Market
- LegendsOfValorGame → LegendsOfValorBoard, HeroUnit, MonsterUnit
- Hero → Inventory
- Tile → Market

MonstersAndHeroesGame has a Board, party, game Market, and collections of heroes, monsters, and items.

LegendsOfValorGame has a LegendsOfValorBoard, HeroUnit, MonsterUnit, Market, and AI behavior objects.

Composition over Inheritance – Game controllers delegate responsibilities to domain objects instead of inheriting behavior.

3) Association

- Party ↔ Hero
- Battle ↔ Hero, Monster
- MonsterBehavior ↔ MonsterUnit
- DamageCalculator ↔ combat entities

HeroUnit associates a Hero with a Lane and Position.

MonsterUnit associates a Monster with its lane and board position.

Represents logical relationships without ownership or tight coupling.

4) Dependency

- Action classes depend on LegendsOfValorGame
- UI depends on GameView
- Boards depend on terrain enums

5) Polymorphism

- Different hero types (Warrior, Sorcerer, Paladin) override shared behaviors such as attacks, stat usage, and equipment handling.
- Different monster types (Dragon, Spirit, Exoskeleton) exhibit varied combat characteristics while sharing a common interface.

Runtime Polymorphism – enables flexible combat and interaction logic without conditional branching.

6) Encapsulation

- Game state (HP, mana, gold, inventory, position) is fully encapsulated within Hero, Monster, Inventory, and Party classes.
- External classes interact only through public methods (e.g., takeDamage, gainExperience, addGold).
- Protects internal state and reduces unintended side effects during gameplay.

7) Factory / Adapter Pattern

- HeroFactoryAdapter is used to load heroes, monsters, and items for Legends of Valor.
- Decouples game logic from concrete object creation and data sources.
- Factory + Adapter Pattern – abstracts object creation and adapts existing classes for reuse in a new game mode.

8) Strategy Pattern

- DamageCalculator and MonsterBehavior are injected into LegendsOfValorGame.
- Allows swapping different combat rules or AI strategies without modifying the game controller.
- Strategy Pattern – enables flexible, interchangeable algorithms for combat and monster behavior.

9) Builder Pattern

- LegendsOfValorBoard.Builder is used to construct the board with configurable terrain distribution and randomness.
- Improves readability and scalability of board creation logic.
- Builder Pattern – simplifies complex object creation with optional parameters.

Scalability

1. **Abstract Game base class**
 - New game modes can be added by extending the Game without changing existing games.
2. **Separated board implementations**
 - Board and LegendsOfValorBoard are independent, allowing different map sizes/layouts.
3. **Action-based hero behavior (LoV)**
 - Adding new actions (e.g., *Defend*, *Special Move*) requires only a new Action class.
4. **Encapsulated combat logic**
 - DamageCalculator and MonsterBehavior allow scaling combat rules without touching game flow.

5. **Data-driven entities**
 - Heroes, monsters, and items are loaded via factories → easy to add more content.
6. **Modular market & inventory**
 - Economy scales by adding new item types without changing game loops.
7. **Enum-based rules**
 - Terrain, lanes, weather scale by adding enum values + handlers.

Extensibility

1. **New game mode**
 - Create class NewGame extends Game and implement abstract methods.
2. **New board type**
 - Add a new board class without modifying existing boards.
3. **New hero/monster types**
 - Extend Hero / Monster and register via factory.
4. **New terrain effects**
 - Add enum value + TerrainEffectFactory mapping.
5. **New combat rules**
 - Swap DamageCalculator implementation.
6. **New AI strategies**
 - Implement a new MonsterBehavior.
7. **New UI (future)**
 - Implement another GameView (e.g., GUI) without changing game logic.
8. **Feature isolation**
 - Changes in Legends of Valor do not affect Monsters & Heroes.

Contribution Of Each Member :

Git Hub Link :

https://github.com/2003Ankita/Legends-of-Valor/tree/Legends_Of_Valor

Ankita Patra, U38177365 :

- Designed and implemented the core architecture of the project by introducing a generic Game base class, enabling a clean, extensible structure that supports multiple game modes while preserving shared gameplay flow.

- Developed the complete Monsters & Heroes game flow, including board generation, hero selection, party movement, battle triggering, market interactions, and win-condition handling, ensuring compliance with the original game specification.
- Implemented the Legends of Valor board system (LegendsOfValorBoard), defining lane-based structure, hero and monster nexus placement, terrain generation (Bush, Cave, Koulou, Obstacle), and inaccessible wall constraints as required by the PDF.
- Contributed significantly to the Legends of Valor game logic, including:
 - (i) Initialization and setup logic (initializeGame)
 - (ii) Hero turn orchestration and round progression
 - (iii) Movement validation (canHeroMoveTo, moveHero)
 - (iv) Monster spawning coordination and board-state updates
 - (v) Win-condition checks across hero and monster nexus interactions
- Integrated shared systems across both game modes, such as inventory management, market buy/sell workflows, battle handling, damage resolution, and consistent user-input validation, ensuring reuse rather than duplication.
- Ensured overall code quality and consistency by refactoring controllers, fixing logic gaps (turn flow, market loops, battle triggers), resolving edge cases, and aligning implementations closely with the design requirements.

Ying Huang, U13787608

- Implemented gameplay mechanics for Legends of Valor, including the complete hero and monster turn execution flow, round progression, and coordination between movement, combat, and spawning phases.
- Developed and integrated core movement rules, ensuring heroes and monsters follow lane restrictions, terrain constraints, blocking rules, and nexus boundaries as specified in the game design.
- Contributed extensively to combat interactions, supporting hero-to-monster and monster-to-hero attacks, damage calculation integration, terrain-aware effects, and proper handling of defeat, rewards, and removal from the board.
- Implemented and refined action handling logic, covering hero actions such as move, attack, cast spell, use potion, teleport, recall, pass, and quit, and ensured each action correctly ends or continues a turn.
- Helped structure and stabilize the turn-based gameplay flow, coordinating hero actions, monster AI turns, regeneration phases, spawning intervals, and round resets to maintain consistent and fair gameplay.

- Assisted in refining win-condition checks, validating hero victory (reaching monster nexus) and monster victory (reaching hero nexus), and ensuring game termination occurs cleanly with appropriate end-game handling.

Xiju Jiang, U03732023

- Developed monster behavior and basic AI logic for Legends of Valor, including automated monster movement, target selection, and attack decision-making during monster turns.
- Contributed to combat resolution logic, helping integrate damage calculation with hero and monster statistics, equipped items (weapons and armor), and terrain-based modifiers to ensure balanced and rule-consistent outcomes.
- Assisted in handling terrain effects during battles, validating that bonuses and penalties from different terrain types are correctly applied and reflected in combat results.
- Participated in testing and debugging gameplay interactions, focusing on edge cases such as simultaneous defeats, blocked movement, invalid targets, and entity removal after combat.
- Helped improve robustness and stability, identifying and fixing issues related to battle flow, monster–hero interactions, and unexpected state transitions during extended gameplay.